Design and Simulation-Based Verification of a Frame-Based Canny Edge Detector in Verilog RTL Using Bit-Exact Golden-Model Comparison

Manda Ramasatyapriya
MTECH VLSI, Amrita Vishwa Vidyapeetham,AMRITAPURI, India · ramasatyapriyamanda@gmail.com

***Abstract*—This paper presents a frame-based Canny edge detector written in Verilog-2001 RTL and verifies it by simulation. A controller steps a 32×32 frame through Gaussian smoothing, Sobel gradient (L1 magnitude, four-sector direction), non-maximum suppression, double thresholding and full iterative 8-connected hysteresis, using only integer arithmetic. The design was simulated in QuestaSim 10.7c on six natural grey-scale images (6,144 pixels) and compared pixel by pixel with an integer Python golden model: there were 0 mismatches and 1,474 edge pixels matched. The edge maps were also compared with OpenCV's Canny operating on the same Gaussian-smoothed image and thresholds (120/40): with a ±1-pixel tolerance the mean precision is 0.996, the mean recall 1.000 and the mean F1 0.998; the exact pixel agreement is 98.93–99.71 % (39 differing pixels out of 6,144). No synthesis, timing, power or hardware results were obtained, and the limitations of the six-image evidence are stated explicitly.**

***Index Terms*—Canny edge detection, Verilog HDL, RTL design, QuestaSim, golden-model verification, hysteresis, simulation-based verification**

I. INTRODUCTION

Edge maps reduce an image to its structural outlines and feed tasks such as segmentation, object detection and feature extraction. The Canny detector [1] is widely used as the reference edge detector: it combines smoothing, gradient computation, non-maximum suppression (NMS), double thresholding and edge tracking by hysteresis. Its neighbourhood operations and data-dependent decisions are expensive on general-purpose processors at high frame rates, which has motivated many FPGA implementations.

Prior hardware work includes parallel and pipelined Canny implementations [2], a distributed block-based detector whose hysteresis thresholds are derived from a gradient-magnitude histogram [3], and an improved algorithm with Otsu-based adaptive thresholds on an FPGA [4]. These works target throughput and resource efficiency. This paper has a different and narrower aim: to describe a complete, readable RTL implementation of the full Canny chain, including exact (iterative) hysteresis, and to show how its correctness can be established by simulation against an independent software model.

The contributions are:

1) A frame-based Verilog-2001 Canny implementation with an explicit stage controller and full iterative 8-connected hysteresis, whose arithmetic uses only integers: a [1 2 1; 2 4 2; 1 2 1]/16 Gaussian, the L1 gradient magnitude and a 106/256 approximation of tan 22.5°.
2) A simulation flow in QuestaSim 10.7c with a self-checking testbench, a bit-accurate integer Python golden model, and an independent quality comparison against OpenCV.
3) Measured results on six images: 0 RTL/golden-model mismatches over 6,144 pixels, and a quantified comparison with OpenCV that separates the tolerance-based metrics from exact pixel agreement.
4) An explicit statement of what was not done (synthesis, timing, power, hardware, broad directed tests), so that the claims remain within the evidence.

II. ALGORITHM AND NUMERIC CONVENTIONS

The processing chain is: (1) Gaussian smoothing; (2) gradient magnitude and direction from the Sobel operator [5]; (3) NMS along the gradient direction; (4) double thresholding into strong and weak candidates; (5) hysteresis that keeps weak pixels connected to strong ones. The numeric conventions below are shared by the RTL and the Python golden model.

- **Input:** 8-bit grey scale, W = H = 32, so N = W·H = 1,024 pixels per frame.
- **Gaussian:** integer kernel [1 2 1; 2 4 2; 1 2 1], rounded by adding 8 and shifting right by 4; replicate padding at the frame border.
- **Gradient:** Gx = (p2+2p5+p8) − (p0+2p3+p6), Gy = (p6+2p7+p8) − (p0+2p1+p2) on the Gaussian output (replicate padding); magnitude M = |Gx|+|Gy| (maximum 2040 for 8-bit input), which avoids a square root.
- **Direction:** four sectors from 256|Gy| < 106|Gx| (sector 0, W/E), 256|Gx| < 106|Gy| (sector 2, N/S), otherwise sector 1 (NW/SE) if Gx and Gy have the same sign and sector 3 (NE/SW) if not; 106/256 ≈ tan 22.5° avoids division and arctangent.
- **NMS:** out-of-frame neighbours count as zero; the centre is kept if it is ≥ the first neighbour (W, NW, N or NE) and > the second (E, SE, S or SW); this asymmetric tie rule avoids double-width ridges on plateaus.
- **Thresholds:** high = 120 and low = 40 on the L1 magnitude scale; class 2 = strong (≥ high), 1 = weak (≥ low), 0 = none.
- **Hysteresis:** a weak pixel with at least one strong 8-neighbour becomes strong; scans are repeated until a scan promotes no pixel, which gives the same result as connected-component closure.

- **Output:** binary map, 0xFF for strong edge pixels and 0x00 otherwise.

III. FRAME-BASED RTL ARCHITECTURE

A. Interface and controller

The top module canny_top has parameters W, H and AW (address width, 10 for 1,024 pixels). Its inputs are clk, an active-low asynchronous reset rst_n, start, pixel_valid, an 8-bit pixel_in and two 11-bit thresholds (threshold_high, threshold_low). Its outputs are busy, done, output_valid, output_addr and an 8-bit output_pixel. In the IDLE state the module accepts one pixel per clock while pixel_valid is high until N pixels are stored; when start is asserted with a complete frame, the thresholds are latched and the controller steps through the states of Fig. 1. The input is therefore loaded first and processed afterwards, in contrast with a streaming pipeline.

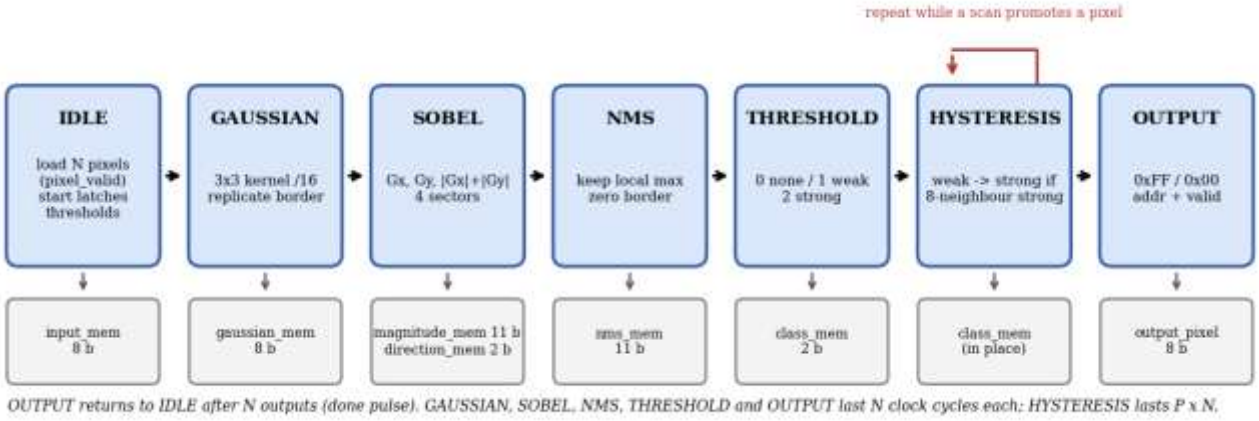


Fig. 1. Controller states and the arrays written by each state. Six memory arrays hold the intermediate frames; the hysteresis state updates the class array in place.

B. Stage implementations

Each stage is a Verilog function evaluated for one pixel per clock and written into a memory array indexed by a pixel counter. The Gaussian state computes the 3×3 weighted sum with replicate-border access and writes gaussian_mem. The Sobel state computes Gx, Gy, the magnitude and the sector and stores them in magnitude_mem and direction_mem. The NMS state reads the two neighbours of the current pixel along its sector, using zero for positions outside the frame, and writes nms_mem. The threshold state classifies each NMS value into class_mem.

The hysteresis state scans the frame in raster order. A weak pixel with a strong 8-neighbour inside the frame is promoted to strong in the same memory, so a promotion can be seen by later pixels of the same scan. At the last pixel the controller starts another scan if any pixel was promoted during the current one; otherwise it moves on. The output state then emits N values with output_valid, output_addr and output_pixel, and raises done on the last one.

C. Storage and cycle count

The declared arrays hold 8 + 8 + 11 + 2 + 11 + 2 = 42 bits per pixel, i.e. 42×1,024 = 43,008 bits for N = 1,024. The controller spends exactly N clock cycles in each of the Gaussian, Sobel, NMS, threshold and output states, and P·N cycles in the hysteresis state, where P is the number of raster scans including the final scan that promotes nothing. Hence the controller time from start to done is (5+P)·N cycles plus a few cycles of handshake. These figures are derived from the RTL structure (Section V-C), not extracted from the waveform; the memory arrays were not synthesised, so how a tool would map them is not known.

IV. VERIFICATION AND EVALUATION METHODOLOGY

A. Test images and vectors

Six images were used: five JPEG files and one PNG (l1.jpg–l5.jpg and l6.png), namely a portrait (Image 1), a photograph of the moon (Image 2), a peppers image (Image 3), a pattern of coloured candies (Image 4), a monarch butterfly on flowers (Image 5) and an espresso cup (Image 6). Each was converted to 8-bit grey scale and resized to 32×32 pixels with Lanczos resampling (Pillow), and written as a HEX file of 1,024 two-digit values in row-major order. Thresholds were fixed at 120/40 for all images.

B. Golden model and testbench

An integer Python/NumPy golden model implements exactly the conventions of Section II, including full iterative hysteresis, and writes a reference edge map as HEX (0xFF/0x00). The Verilog testbench (tb_canny_top.v) reads one input frame and its reference map, resets the design, streams 1,024 pixels at one per clock, pulses start, waits for done, writes every output pixel to rtl_imageN_output.hex and counts mismatches against the reference by output address. This is repeated for the six images, with a reset before each. The simulation was run in QuestaSim 10.7c; a waveform database and a waveform layout (Questa .wlf and .do files) were saved for the run.

C. Quality comparison and cross-check

To obtain evidence of algorithmic correctness that does not depend on the authors' own model, the RTL output is compared with OpenCV [6] (cv2.Canny, aperture 3, L1 gradient) applied to the same Gaussian-smoothed image with the same thresholds, so the comparison concerns the gradient,

NMS and hysteresis stages. Precision is the fraction of RTL edge pixels that have an OpenCV edge pixel within ±1 pixel (3×3 dilation), recall is the converse, and F1 is their harmonic mean; this symmetric dilation matching is not a one-to-one assignment. Exact pixel agreement is the fraction of pixels whose binary value is identical, with no tolerance.

As an additional cross-check performed for this paper, the RTL output files were compared with a second Python implementation written separately from the primary golden model (connected-component labelling from SciPy [7] for hysteresis). It was run on the same input HEX files and its maps were compared with both the reference HEX files and the RTL output files; it also reproduced the metrics of Table II. The number of hysteresis scans P of Table III was obtained by emulating the controller's in-place raster scan in Python.

V. RESULTS

A. Functional verification

All six frames passed (Table I). Over 6,144 compared pixels there were 0 mismatches between the RTL output and the golden model, and the 1,474 RTL edge pixels equal the 1,474 edge pixels of the model. The separately written second model also matched the RTL output and the reference HEX files on all six images. Because the golden model was written for this design, bit-exactness shows that the RTL implements the intended algorithm; it does not by itself show that the algorithm is correct, which is the purpose of the OpenCV comparison. The testbench compares only the final edge map, so intermediate stage arrays were not checked individually.

B. Edge quality against OpenCV

With the ±1-pixel tolerance (Table II) the mean precision is 0.996, the mean recall 1.000 and the mean F1 0.998; recall is 1.000 for every image and the lowest precision is 0.981 (Image 6). The exact comparison is less flattering and more informative: 39 of 6,144 pixels differ (30 set only in the RTL output and 9 set only in OpenCV), i.e. exact agreement of 98.93–99.71 % per image (mean 99.37 %). Only 4 of the 39 differing pixels lie on the outer ring of the frame. The RTL produces 1,474 edge pixels against 1,453 for OpenCV, so the RTL output is slightly denser. Fig. 3 shows where the differences occur.

The tolerance-based scores must be read with care: at 32×32 the edge maps are dense (10.84–31.84 % of pixels), so almost any RTL edge pixel has an OpenCV edge pixel within one pixel and the metrics saturate near 1. The exact agreement and the counts of differing pixels are the more discriminating figures. The measured maximum gradient magnitude after Gaussian smoothing and before NMS lies between 356 and 756, well below the 11-bit limit of 2040.

C. Hysteresis scans and controller cycles

Table III lists the number of strong and weak pixels after thresholding and the number P of hysteresis scans, which ranges from 2 to 9. The controller time from start to done is (5+P)·N cycles, between 7,168 and 14,336 cycles for these frames. The table also gives the number of edge pixels a single hysteresis scan would produce in the golden model: for Image 5 it is 257 against 295 with full hysteresis, which shows why iterative promotion matters on images with long weak edge chains. Cycle counts are derived from the RTL structure and emulation, not measured on the waveform.

TABLE I. RTL against the Python golden model (QuestaSim 10.7c, thresholds 120/40, 32×32 frames).

Image	Source file	Python edge px	RTL edge px	Mismatches	Pixel agreement	Result
1 (portrait)	l1.jpg	301	301	0	100.00 %	PASS
2 (moon)	l2.jpg	173	173	0	100.00 %	PASS
3 (peppers)	l3.jpg	326	326	0	100.00 %	PASS
4 (candies)	l4.jpg	111	111	0	100.00 %	PASS
5 (butterfly)	l5.jpg	295	295	0	100.00 %	PASS
6 (espresso)	l6.png	268	268	0	100.00 %	PASS
Total		**1,474**	**1,474**	**0**	**100.00 %**	**6/6 PASS**

TABLE II. RTL against OpenCV Canny on the same Gaussian-smoothed image. Precision, recall and F1 use ±1 px tolerance; "Exact mismatches" counts differing pixels (RTL-only / OpenCV-only); agreement is exact. Last column: maximum |Gx|+|Gy| before NMS.

Image	RTL edge px	OpenCV edge px	Edge density	Precision	Recall	F1	Exact mismatches	Exact agreement	Max grad.
1	301	298	29.39 %	0.997	1.000	0.998	7 (5 / 2)	99.32 %	570
2	173	172	16.89 %	1.000	1.000	1.000	3 (2 / 1)	99.71 %	740
3	326	320	31.84 %	0.997	1.000	0.998	6 (6 / 0)	99.41 %	646
4	111	111	10.84 %	1.000	1.000	1.000	4 (2 / 2)	99.61 %	510
5	295	291	28.81 %	1.000	1.000	1.000	8 (6 / 2)	99.22 %	356
6	268	261	26.17 %	0.981	1.000	0.991	11 (9 / 2)	98.93 %	756
Mean / total	**1,474**	**1,453**	**23.99 %**	**0.996**	**1.000**	**0.998**	**39 (30 / 9)**	**99.37 %**	**356–756**

TABLE III. Hysteresis behaviour and controller cycles (N = 1,024). P counts raster scans including the final scan without promotions. Cycles are derived from the controller structure and an emulation of its scan, not read from the waveform; the last two columns come from the golden model.

Image	Strong px	Weak px	Scans P	Hysteresis cycles P·N	Controller cycles (5+P)·N	Edge px, single scan	Edge px, full hyst.
1	279	32	9	9,216	14,336	289	301
2	143	53	3	3,072	8,192	168	173
3	305	25	3	3,072	8,192	324	326
4	109	29	2	2,048	7,168	111	111
5	200	136	5	5,120	10,240	257	295
6	220	65	8	8,192	13,312	240	268

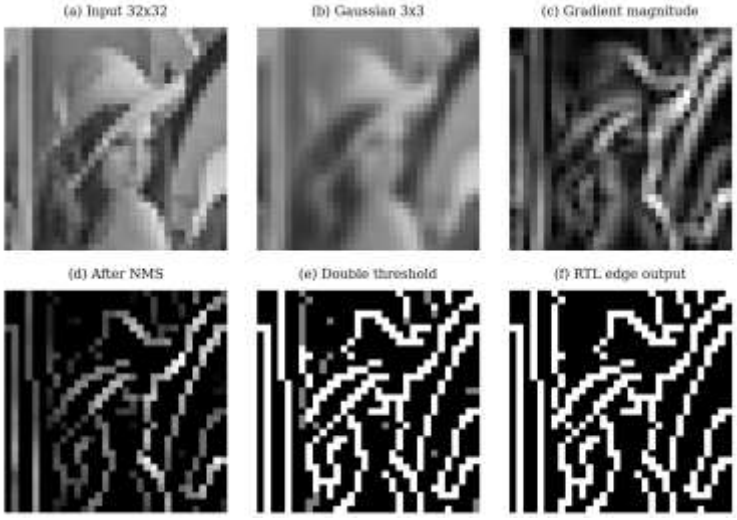


Fig. 2. Stage-by-stage outputs for Image 1 (thresholds 120/40): (a) input, (b) Gaussian, (c) gradient magnitude and (d) after NMS (display-scaled; golden-model stages), (e) double threshold (grey = weak, white = strong), (f) final RTL edge output.

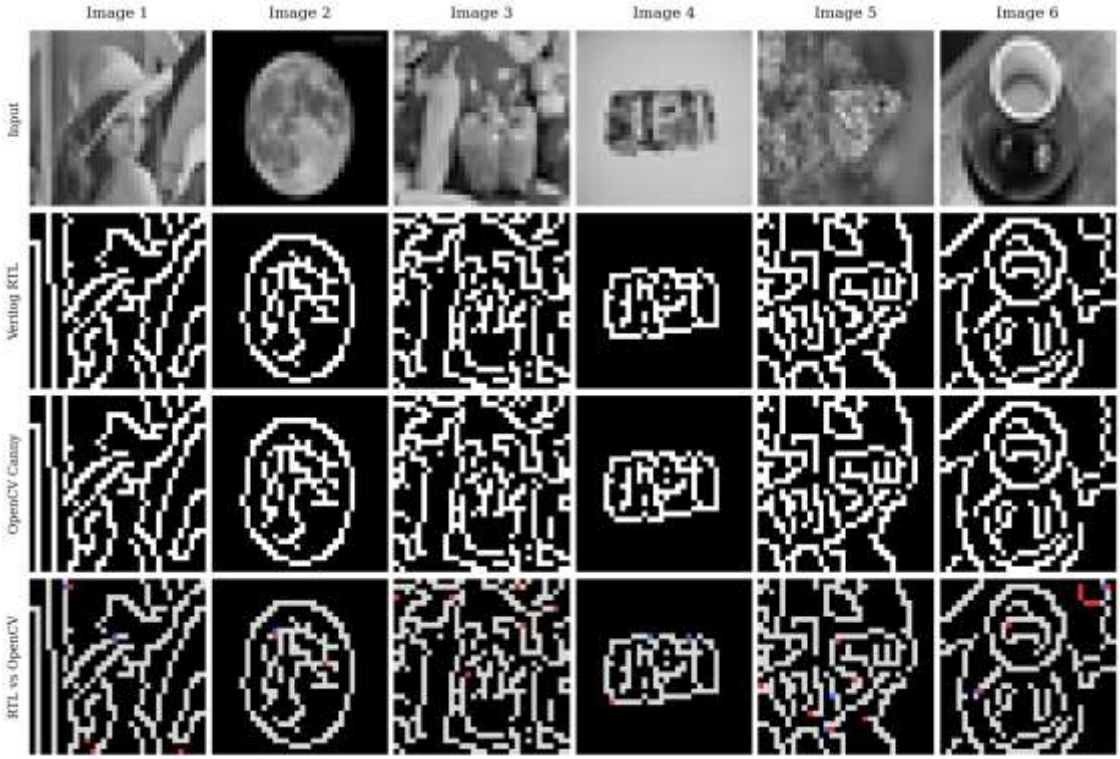


Fig. 3. Six test images: input, Verilog RTL output, OpenCV Canny, and their difference (grey = both, red = RTL only, blue = OpenCV only). The RTL output is identical to the Python golden model on every image.

VI. DISCUSSION AND LIMITATIONS

- **Frame-based, not streaming.** The design stores the whole frame and the controller reads and writes one pixel per clock in each state, so a frame needs (5+P)·N controller cycles rather than one pixel per cycle. This is simple to verify but is not a real-time architecture; a line-buffer streaming version is future work.
- **Not a set of stand-alone modules.** The stages are Verilog functions and controller states inside one module; they are logically separated but not independent, reusable modules.
- **No synthesis or hardware evidence.** No synthesis, area, timing, maximum frequency, power or FPGA results were obtained. The RTL uses behavioural constructs (constant multiplications, division and modulo by W, loops inside functions), and whether and how efficiently it synthesises was not evaluated.
- **Small test set.** Six natural images at 32×32 and a single threshold pair (120/40, not tuned) were used. Directed tests (constant image, single impulse, step edges, checkerboard, random noise, odd dimensions), back-to-back frames without reset and other boundary cases were not run. The DUT was reset before each image.
- **Limited observability.** Only the final edge map is compared in the testbench, and the comparison with OpenCV uses a lenient ±1-pixel tolerance on dense maps. The OpenCV reference is a software baseline, and differences can come from implementation details such as numerical conventions and edge linking; the cause of the individual differing pixels was not analysed.
- **Waveform.** A waveform database and layout were saved; no waveform screenshot is included in this paper.
- **Source images.** The input photographs were downsampled to 32×32 with Lanczos resampling, which discards most fine structure; results on full-resolution images are not claimed.

VII. CONCLUSION

A frame-based Canny edge detector with exact iterative hysteresis was implemented in Verilog RTL and simulated in QuestaSim 10.7c. On six 32×32 images its output matched a bit-accurate integer golden model with 0 mismatches over 6,144 pixels. Against OpenCV Canny on the same smoothed image, the mean precision, recall and F1 with ±1-pixel tolerance were 0.996, 1.000 and 0.998, and the exact pixel agreement was 99.37 % on average. Future work includes directed and random verification, larger images and several threshold pairs, a modular streaming architecture with line buffers, synthesis and timing analysis, and FPGA validation.

REFERENCES

[1] J. Canny, “A computational approach to edge detection,” IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.

[2] C. Gentsos, C. L. Sotiropoulou and S. Nikolaidis, “Real-time Canny edge detection parallel implementation for FPGAs,” in Proc. 17th IEEE Int. Conf. Electronics, Circuits and Systems (ICECS), Athens, Greece, Dec. 2010, pp. 499–502.

[3] Q. Xu, S. Varadarajan, C. Chakrabarti and L. J. Karam, “A distributed Canny edge detector: Algorithm and FPGA implementation,” IEEE Trans. Image Process., vol. 23, no. 7, pp. 2944–2960, Jul. 2014, doi: 10.1109/TIP.2014.2311656.

[4] L. Guo and S. Wu, “FPGA implementation of a real-time edge detection system based on an improved Canny algorithm,” Appl. Sci., vol. 13, no. 2, Art. no. 870, 2023, doi: 10.3390/app13020870.

[5] I. Sobel and G. Feldman, “A 3×3 isotropic gradient operator for image processing,” presented at the Stanford Artificial Intelligence Project (SAIL), 1968.

[6] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.

[7] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.